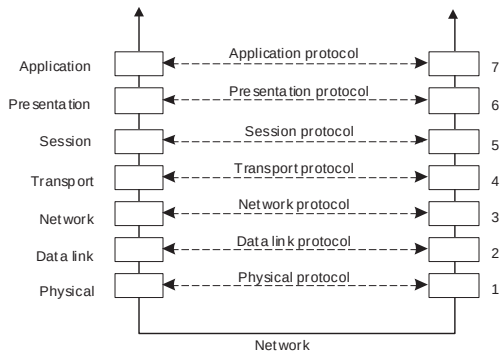


并行与分布式计算

-- 通信

基本网络模型



缺点

- ▶ 仅关注消息传递
- ▶ 通常是不需要或不需要的功能
- ▶ 违反访问透明度

低层级

回顾

- ▶ **Physical layer**物理层：包含位的规范和实现，以及它们在发送方和接收方之间的传输
- ▶ **Data link layer**数据链路层：规定将一系列位传输到帧中以允许错误和流量控制
- ▶ **Network layer**网络层：描述如何在计算机网络中路由数据包。

观察

对于许多分布式系统，最低级别的接口是网络层。

传输层

Important

传输层为大多数分布式系统提供实际的通信设施。

标准互联网协议

- ▶ TCP : 面向连接、可靠、面向流的通信
- ▶ UDP : 不可靠（尽力而为）数据报通信

中间件层

观察

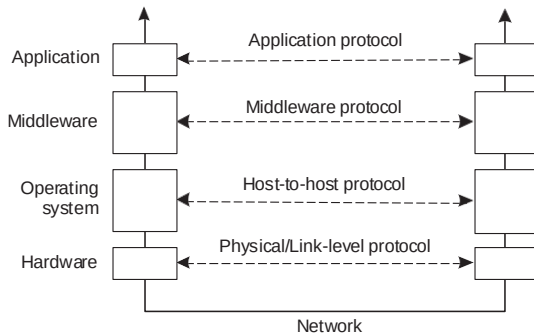
中间件的发明是为了提供可供许多不同应用程序使用的通用服务和协议
一组丰富的通信协议

- ▶ 一组丰富的通信协议
- ▶ 数据（解）编组，集成系统所必需的
- ▶ 命名协议，允许轻松共享资源
- ▶ 安全通信的安全协议
- ▶ 扩展机制，例如用于复制和缓存

Note

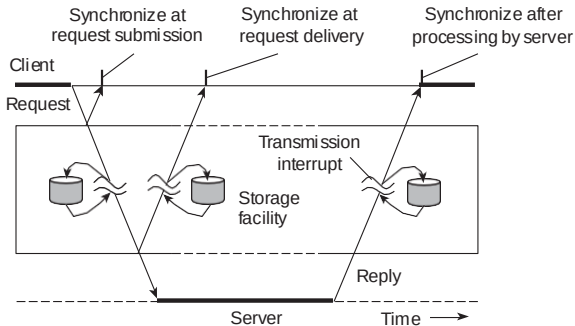
剩下的是真正特定于应用程序的协议.....例如？

经过调整的分层方案



通信类型

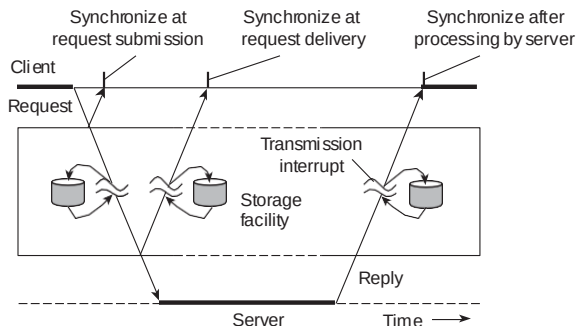
区别...



- ▶ 瞬时通信与持久通信
- ▶ 异步通信与同步通信

通信类型

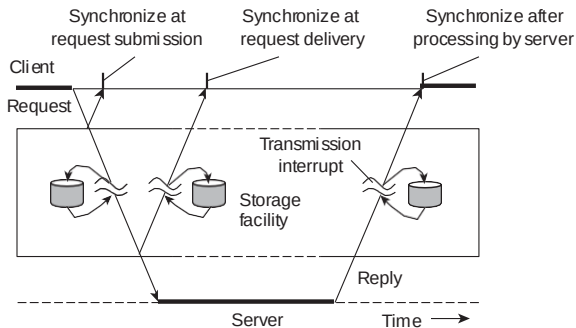
瞬时与持久



- ▶ **Transient communication** 瞬时通信：当消息无法传递到下一个服务器或接收方时，通信服务器会丢弃该消息。
- ▶ **Persistent communication** 持久通信：只要需要传递消息，消息就会存储在通信服务器上。

通信类型

同步点



- ▶ 请求提交时
- ▶ 请求交付时
- ▶ 请求处理后

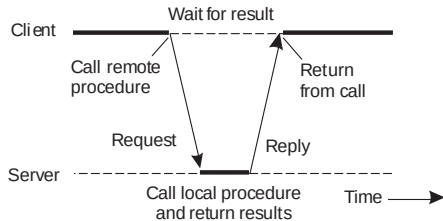
基本RPC操作

观察

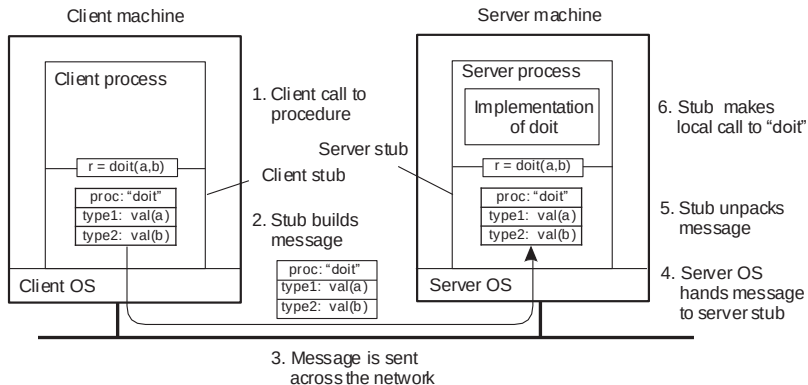
- ▶ 应用程序开发人员熟悉简单的程序模型
- ▶ 精心设计的程序独立运行（黑盒）
- ▶ 没有根本原因不在单独的机器上执行程序

结论

可以使用过程调用机制来隐藏调用者和被调用者之间的通信。



Basic RPC operation



1. Client procedure calls client stub.
2. Stub builds message; calls local OS.
3. OS sends message to remote OS.
4. Remote OS gives message to stub

6. Server does local call; returns result to stub.
7. Stub builds message; calls OS.
8. OS sends message to client's OS.
9. Client's OS gives message to stub.
10. Client stub unpacks result; returns to

RPC: 参数解析

不仅仅是将参数包装到消息中

- ▶ 客户端和服务端机器可能具有不同的数据表示形式（想想字节顺序）
- ▶ 包装参数意味着将值转换为字节序列
- ▶ 客户端和服务端必须就相同的编码达成一致：
- ▶ 如何表示基本数据值（整数、浮点数、字符）
- ▶ 如何表示复杂数据值（数组、联合）

结论

客户端和服务端需要正确地解释消息，并将其转换成与机器相关的表示形式。

RPC: 参数解析

假设

- ▶ 复制输入/复制输出语义：执行过程时，不能假设任何参数值。
- ▶ 所有要操作的数据都通过参数传递。不包括传递对（全局）数据的引用。

RPC: 参数解析

假设

- ▶ 复制输入/复制输出语义：执行过程时，不能假设任何参数值。
- ▶ 所有要操作的数据都通过参数传递。不包括传递对（全局）数据的引用。

结论

无法实现完全的访问透明。

RPC: 参数解析

假设

- ▶ 复制输入/复制输出语义：执行过程时，不能假设任何参数值。
- ▶ 所有要操作的数据都通过参数传递。不包括传递对（全局）数据的引用。

结论

无法实现完全的访问透明。

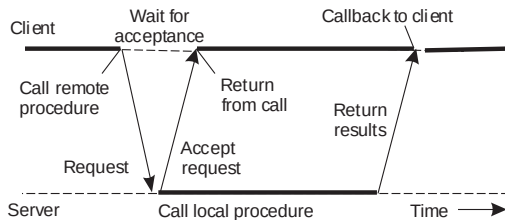
远程引用机制增强了访问透明度

- ▶ 远程引用提供对远程数据的统一访问
- ▶ 远程引用可以作为 **RPC** 中的参数传递
- ▶ 注意：存根有时可以用作此类引用

异步 RPCs

本质

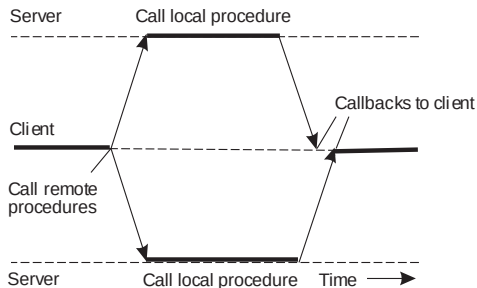
尝试摆脱严格的请求-答复行为，但让客户端继续，而无需等待服务器的答复。



发送多个RPCs

Essence本质

发送RPC请求到一组服务器



消息

面向消息的中间件

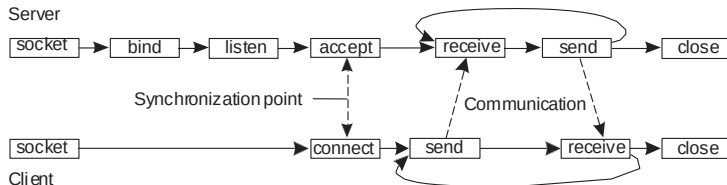
旨在实现高级持久异步通信：

- ▶ 进程之间相互发送消息，这些消息被排队
- ▶ 发送者无需等待立即回复，但可以做其他事情
- ▶ 中间件通常可以确保容错能力

瞬时消息传递：套接字

Berkeley 套接字接口

Operation	Description
socket	Create a new communication end point
bind	Attach a local address to a socket
listen	Tell operating system what the maximum number of pending connection requests should be
accept	Block caller until a connection request arrives
connect	Actively attempt to establish a connection
send	Send some data over the connection
receive	Receive some data over the connection
close	Release the connection



Sockets: Python code

Server

```
1 from socket import *
2 s = socket(AF_INET, SOCK_STREAM)
3 s.bind((HOST, PORT))
4 s.listen(1)
5 (conn, addr) = s.accept() # returns new socket and addr. client
6 while True:               # forever
7     data = conn.recv(1024) # receive data from client
8     if not data: break     # stop if client stopped
9     conn.send(str(data)+"*") # return sent data plus an "*"
10 conn.close()              # close the connection
```

Client

```
1 from socket import *
2 s = socket(AF_INET, SOCK_STREAM)
3 s.connect((HOST, PORT)) # connect to server (block until accepted)
4 s.send('Hello, world') # send same data
5 data = s.recv(1024)    # receive the response
6 print data              # print the result
7 s.close()               # close the connection
```

使套接字更易于使用

观察

套接字是相当低级的，编程时很容易出错。但是，它们的使用方式通常是相同的（例如在客户端-服务器设置中）。

替代方案：ZeroMQ

通过配对套接字提供更高级别的表达：一个用于在进程 **P** 上发送消息，另一个用于在进程 **Q** 上接收消息。所有通信都是异步的。

三种模式

- ▶ 请求-回复
- ▶ 发布-订阅
- ▶ 管道

Request-reply

Server

```
1 import zmq
2 context = zmq.Context()
3
4 p1 = "tcp://" + HOST + ":" + PORT1 # how and where to connect
5 p2 = "tcp://" + HOST + ":" + PORT2 # how and where to connect
6 s = context.socket(zmq.REP) # create reply socket
7
8 s.bind(p1) # bind socket to address
9 s.bind(p2) # bind socket to address
10 while True:
11     message = s.recv() # wait for incoming message
12     if not "STOP" in message: # if not to stop...
13         s.send(message + "*") # append "*" to message
14     else: # else...
15         break # break out of loop and end
```

Request-reply

Client

```
1 import zmq
2 context = zmq.Context()
3
4 php = "tcp://" + HOST + ":" + PORT # how and where to connect
5 s = context.socket(zmq.REQ) # create socket
6
7 s.connect(php) # block until connected
8 s.send("Hello World") # send message
9 message = s.recv() # block until response
10 s.send("STOP") # tell server to stop
11 print message # print result
```

Publish-subscribe

Server

```
1 import zmq, time
2
3 context = zmq.Context()
4 s = context.socket(zmq.PUB)           # create a publisher socket
5 p = "tcp://" + HOST + ":" + PORT     # how and where to communicate
6 s.bind(p)                            # bind socket to the address
7 while True:
8     time.sleep(5)                    # wait every 5 seconds
9     s.send("TIME" + time.asctime())  # publish the current time
```

Client

```
1 import zmq
2
3 context = zmq.Context()
4 s = context.socket(zmq.SUB)           # create a subscriber socket
5 p = "tcp://" + HOST + ":" + PORT     # how and where to communicate
6 s.connect(p)                          # connect to the server
7 s.setsockopt(zmq.SUBSCRIBE, "TIME")  # subscribe to TIME messages
8
9 for i in range(5): # Five iterations
10     time = s.recv() # receive a message
11     print time
```


Pipeline

Source

```
1 import zmq, time, pickle, sys, random
2
3 context = zmq.Context()
4 me = str(sys.argv[1])
5 s = context.socket(zmq.PUSH)           # create a push socket
6 src = SRC1 if me == '1' else SRC2      # check task source host
7 prt = PORT1 if me == '1' else PORT2    # check task source port
8 p = "tcp://" + src + ":" + prt         # how and where to connect
9 s.bind(p)                             # bind socket to address
10
11 for i in range(100):                   # generate 100 workloads
12     workload = random.randint(1, 100)  # compute workload
13     s.send(pickle.dumps((me, workload))) # send workload to worker
```

Pipeline

Worker

```
1 import zmq, time, pickle, sys
2
3 context = zmq.Context()
4 me = str(sys.argv[1])
5 r = context.socket(zmq.PULL)           # create a pull socket
6 p1 = "tcp://" + SRC1 + ":" + PORT1     # address first task source
7 p2 = "tcp://" + SRC2 + ":" + PORT2     # address second task source
8 r.connect(p1)                         # connect to task source 1
9 r.connect(p2)                         # connect to task source 2
10
11 while True:
12     work = pickle.loads(r.recv())      # receive work from a source
13     time.sleep(work[1]*0.01)           # pretend to work
```

MPI : 当需要很大的灵活性时

代表性操作

Operation	Description
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until transmission starts
MPI_sendrecv	Send a message and wait for reply
MPI_issend	Pass reference to outgoing message, and continue
MPI_jssend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

消息中间件

本质

通过支持中间件级队列实现异步持久通信。队列对应于通信服务器上的缓冲区。

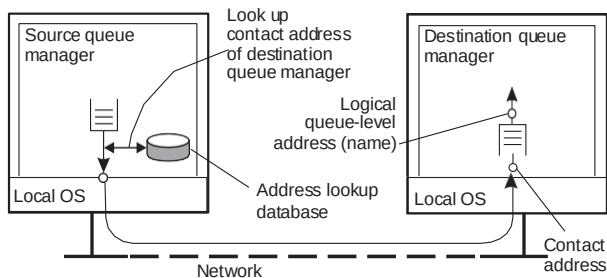
操作

Operation	Description
put	Append a message to a specified queue
get	Block until the specified queue is nonempty, and remove the first message
poll	Check a specified queue for messages, and remove the first. Never block
notify	Install a handler to be called when a message is put into the specified queue

通用模型

队列管理器

队列由**队列管理器**管理。应用程序只能将消息放入本地队列。
获取消息只能通过从**本地**队列中提取消息⇒队列管理器需要路由消息



消息代理

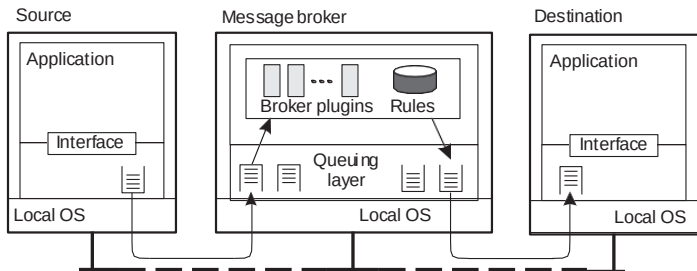
观察

消息队列系统假设一个[通用的消息传递协议](#)：所有应用程序都同意消息格式（即结构 and 数据表示）

代理处理队列管理器系统中的应用程序异构性

- ▶ 将传入消息转换为目标格式
- ▶ 通常充当[应用程序网关](#)
- ▶ 可提供基于[主题](#)的路由功能（即[发布-订阅](#)功能）

消息代理：通用架构



IBM WebSphere 消息队列系统

基本概念

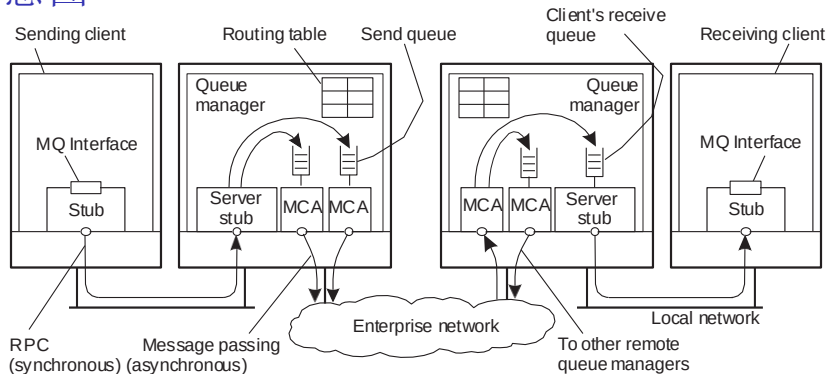
- ▶ 应用程序特定的消息被放入队列并从队列中删除
- ▶ 队列位于队列管理器的管理之下
- ▶ 进程只能将消息放入本地队列，或通过 RPC 机制放入队列

消息转换

- ▶ 消息在队列之间传输
- ▶ 消息在不同进程的队列之间传输，需要一个通道channel
- ▶ 通道的每个端点都有一个消息通道代理
- ▶ 消息通道代理负责：
 - ▶ 使用较低级别的网络通信设施（例如 TCP/IP）设置通道
 - ▶ 从传输层数据包中/在传输层数据包中解包消息
 - ▶ 发送/接收数据包

IBM WebSphere 消息队列系统

示意图



- ▶ 通道本质上是单向的
- ▶ 消息到达时自动启动 **MCA**
- ▶ 可以创建任何队列管理器网络
- ▶ 路由是手动设置的（系统管理）

消息通道代理

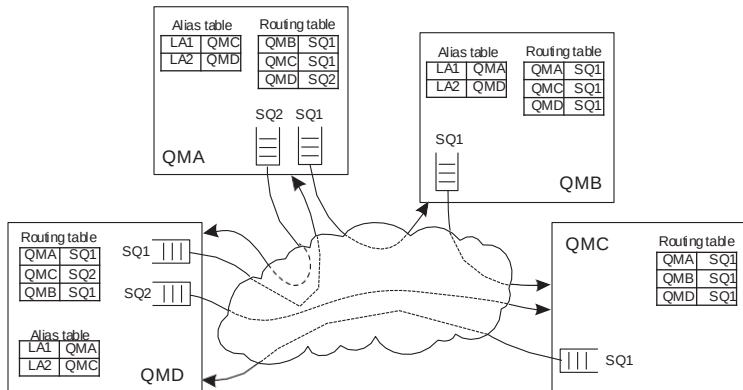
与消息通道代理相关的一些属性

Attribute	Description
Transport type	Determines the transport protocol to be used
FIFO delivery	Indicates that messages are to be delivered in the order they are sent
Message length	Maximum length of a single message
Setup retry count	Specifies maximum number of retries to start up the remote MCA
Delivery retries	Maximum times MCA will try to put received message into queue

IBM WebSphere 消息队列系统

路由

通过使用逻辑名称，结合对本地队列的名称解析，可以将消息放入远程队列



Application-level multicasting

应用级多播

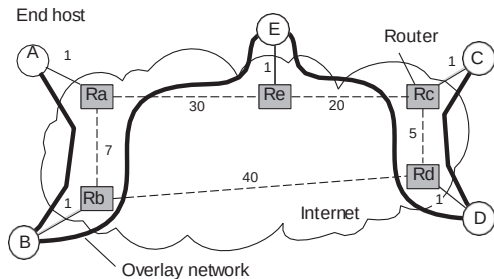
本质

将分布式系统的节点组织成覆盖网络，并使用该网络传播数据：

- ▶ 通常是一棵树，通向唯一的路径
- ▶ 或者，也可以是网状网络，需要某种形式的路由

性能分析

不同衡量标准



- ▶ **Link stress** 链路压力: ALM 消息经过相同物理链路的频率是多少? 例如: 从 A 到 D 的消息需要经过(Ra, Rb) 两次。
- ▶ **Stretch** 拉伸: ALM 级路径和网络级路径之间的延迟比率。例如: 消息 B 到 C 在 ALM 处遵循长度为 73 的路径, 但在网络级遵循长度为 47 的路径 $\Rightarrow$ 拉伸 = $73/47$ 。

Flooding 洪泛

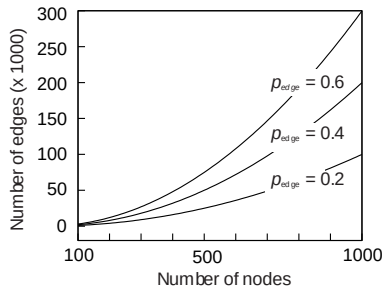
Essence 本质

P 只需向其每个邻居发送一条消息 m 。每个邻居都会转发该消息，但 P 除外，并且前提是它之前从未见过 m 。

性能

边越多越贵！

随机覆盖的大小与节点数的关系



Flooding 洪泛

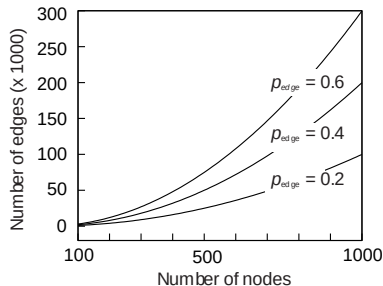
Essence 本质

P 只需向其每个邻居发送一条消息 m 。每个邻居都会转发该消息，但 P 除外，并且前提是它之前从未见过 m 。

性能

边越多越贵！

随机覆盖的大小与节点数的关系



变体

让 Q 以一定的概率 p_{flood} 转发一条消息，甚至可能取决于其自身的邻居数量（即节点度）或其邻居的度。

Epidemic protocols流行病协议

假设没有写-写冲突

- ▶ 更新操作在单个服务器上执行
- ▶ 副本仅将更新状态传递给少数邻居
- ▶ 更新传播是惰性的，即不是即时的
- ▶ 最终，每个更新都应该到达每个副本

两种形式的流行病

- ▶ **Anti-entropy**:反熵：每个副本定期随机选择另一个副本，并交换状态差异，导致之后两个副本的状态相同
- ▶ **Rumor spreading**: 谣言传播：刚刚更新（即已被污染）的副本将其更新告知许多其他副本（也污染了它们）。

Anti-entropy 反熵模型

基本操作

- ▶ 节点 P 从系统中随机选择另一个节点 Q 。
- ▶ **Pull** 拉取: P 仅从 Q 拉取新更新
- ▶ **Push** 推送: P 仅将自己的更新推送到 Q
- ▶ **Push-pull** 推送-拉取: P 和 Q 相互发送更新

观察

对于推拉, 需要 $O(\log(N))$ 轮才能将更新传播到所有 N 个节点 (轮数 = 每个节点都主动开始交换时)。

反熵: 分析

基础知识

考虑一个单一来源，传播其更新。设 p_i 为节点在第 i 轮之后未收到更新的概率

分析：保持无知

- 通过pull拉动, $p_{i+1} = (p_i)^2$:
该节点在第 i 轮中未更新，并且应该在下一轮中联系另一个未知节点。

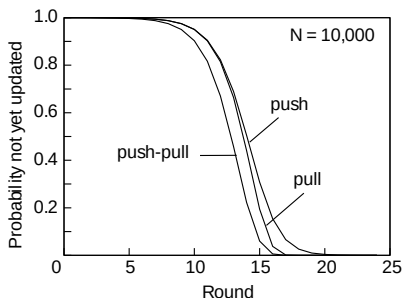
- 通过push推动, $p_{i+1} =$

$$p_i \left(1 - \frac{1}{N-1}\right)^{(N-1)(1-p_i)} \approx p_i e^{-1}$$

(针对小 p_i 以及大 N):

该节点在第 i 轮中处于未知状态，并且在下一轮中没有更新的节点选择联系它。

- 通过 push-pull 推动-拉动: $(p_i)^2 \cdot (p_i e^{-1})$



谣言传播

基本模型

有更新需要报告的服务器 S 将联系其他服务器。如果联系的服务器已经传播了更新，则 S 会以概率 p_{stop} 停止联系其他服务器。

观察

如果 s 是无知服务器（即不知道更新的服务器）的比例，则可以证明，对于许多服务器

$$s = e^{-(1/p_{stop} + 1)(1-s)}$$

正式分析

符号

让 s 表示尚未更新的节点比例（即易受影响的节点）； i 表示已更新（受感染）和活跃节点的比例； r 表示放弃（删除）的更新节点比例。

从流行病理论来看

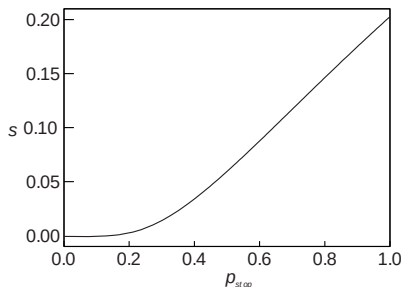
$$\begin{aligned}
 (1) \quad ds/dt &= -s \\
 (2) \quad di/dt &\stackrel{!}{=} s \cdot i - p_{stop} \cdot (1-s) \cdot \\
 \Rightarrow di/ds &\stackrel{!}{=} -(1+p_{stop}) \cdot \frac{p_{stop}}{s} \\
 \Rightarrow i(s) &\stackrel{!}{=} -(1+p_{stop}) \cdot s + p_{stop} \cdot \ln(s) + C
 \end{aligned}$$

Wrapup

$i(1) = 0 \Rightarrow C = 1 + p_{stop} \Rightarrow i(s) = (1 + p_{stop}) \cdot (1-s) + p_{stop} \cdot \ln(s)$. 寻找 $i(s) = 0$, which leads to $s = e^{-(1/p_{stop}+1)(1-s)}$

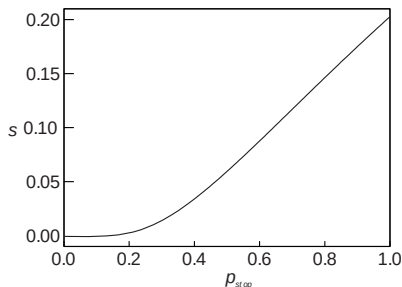
谣言传播

The effect of stopping



Consider 10,000 nodes		
$1/p_{stop}$	s	N_s
1	0.203188	2032
2	0.059520	595
3	0.019827	198
4	0.006977	70
5	0.002516	25
6	0.000918	9
7	0.000336	3

谣言传播



Consider 10,000 nodes		
$1/p_{stop}$	s	N_s
1	0.203188	2032
2	0.059520	595
3	0.019827	198
4	0.006977	70
5	0.002516	25
6	0.000918	9
7	0.000336	3

注意

如果我们真的要确保所有服务器最终都更新，单靠散布谣言是不够的

删除数据

基本问题

我们无法从服务器中删除旧值并期望删除会传播。相反，单纯的删除将在适当的时候使用流行算法撤消

解决方案

必须通过插入[死亡证明](#)将删除注册为特殊更新

删除数据

何时删除死亡证明（不允许永久保留）

- ▶ 运行全局算法来检测是否到处都知道删除，然后收集死亡证明（看起来像垃圾收集）
- ▶ 假设死亡证明在有限的时间内传播，并为证书关联最大寿命（可以冒着无法到达所有服务器的风险来完成）

注意

删除必须真正到达所有服务器。